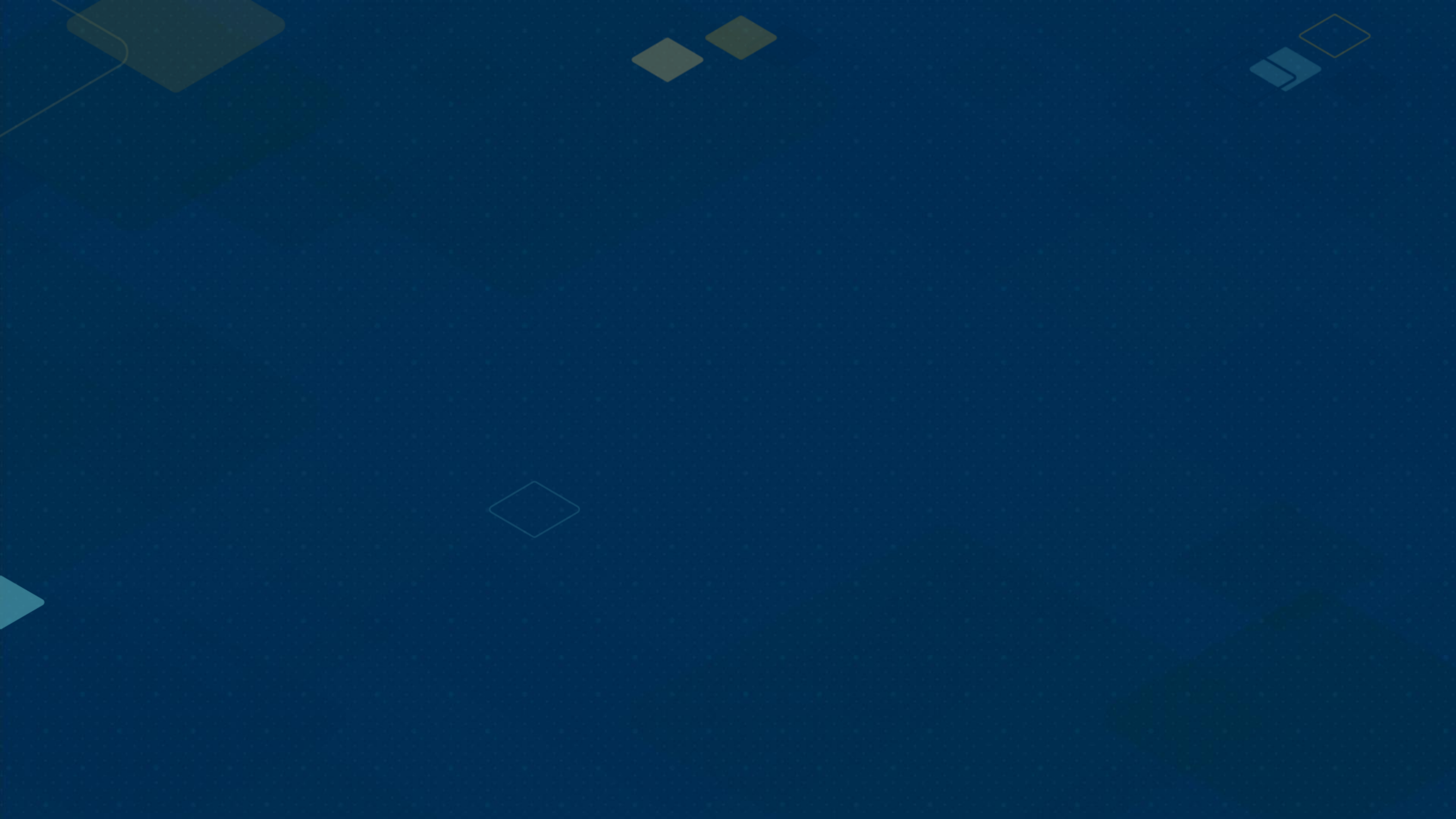




Introduction to BuzzDB







BuzzDB

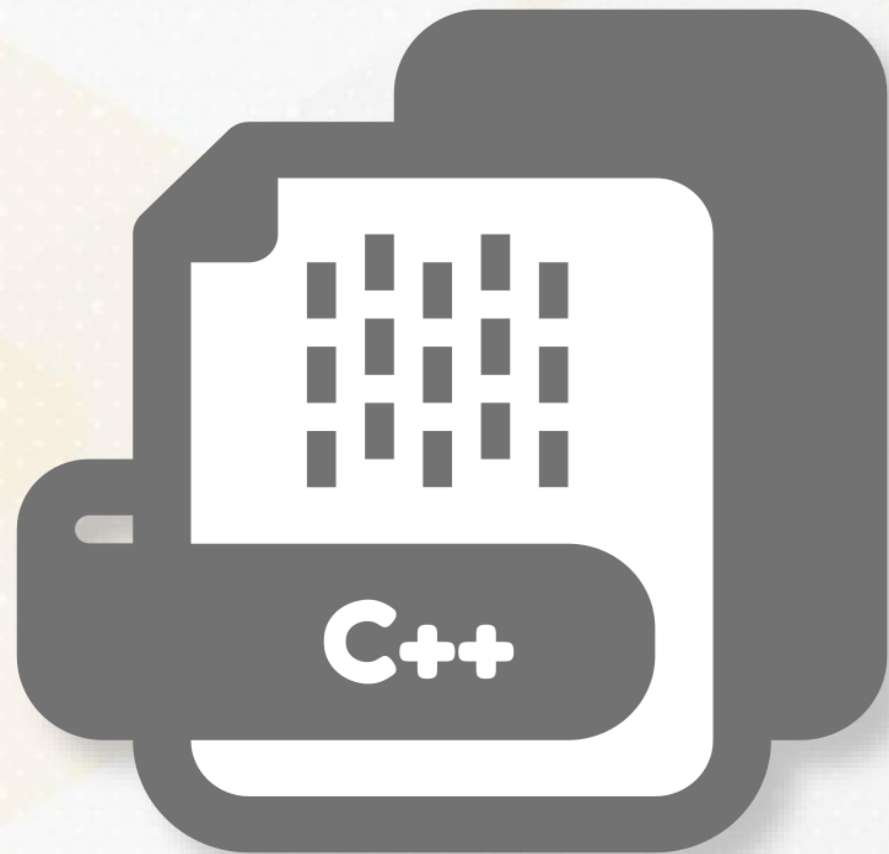


BuzzDB

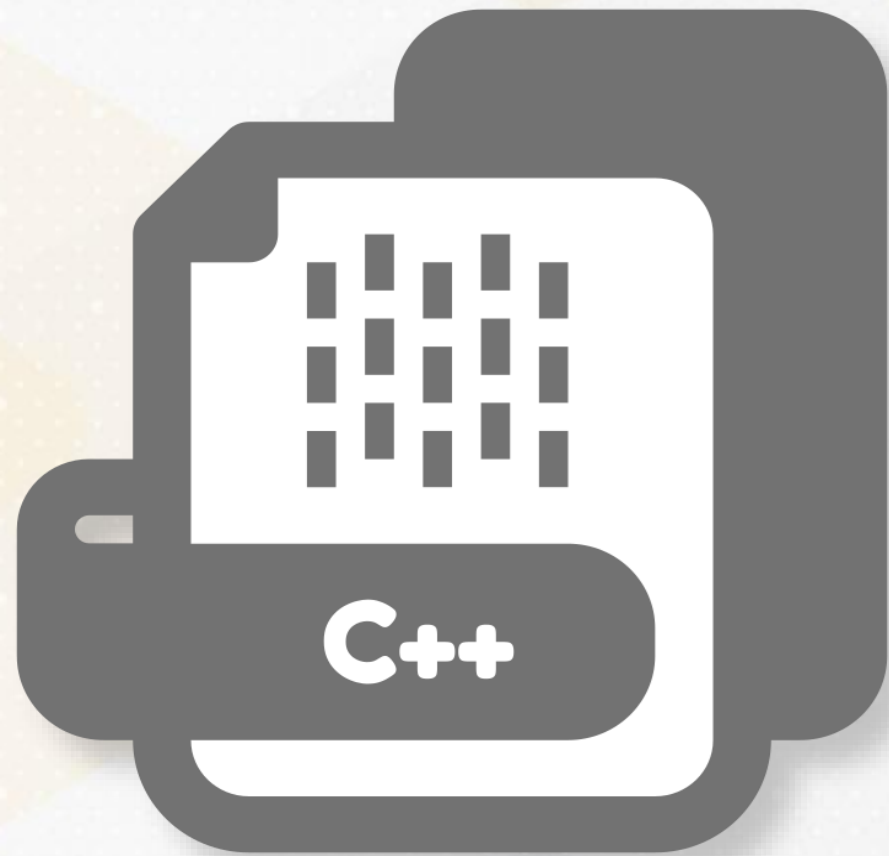
**Fundamentals
of C++**

BuzzDB

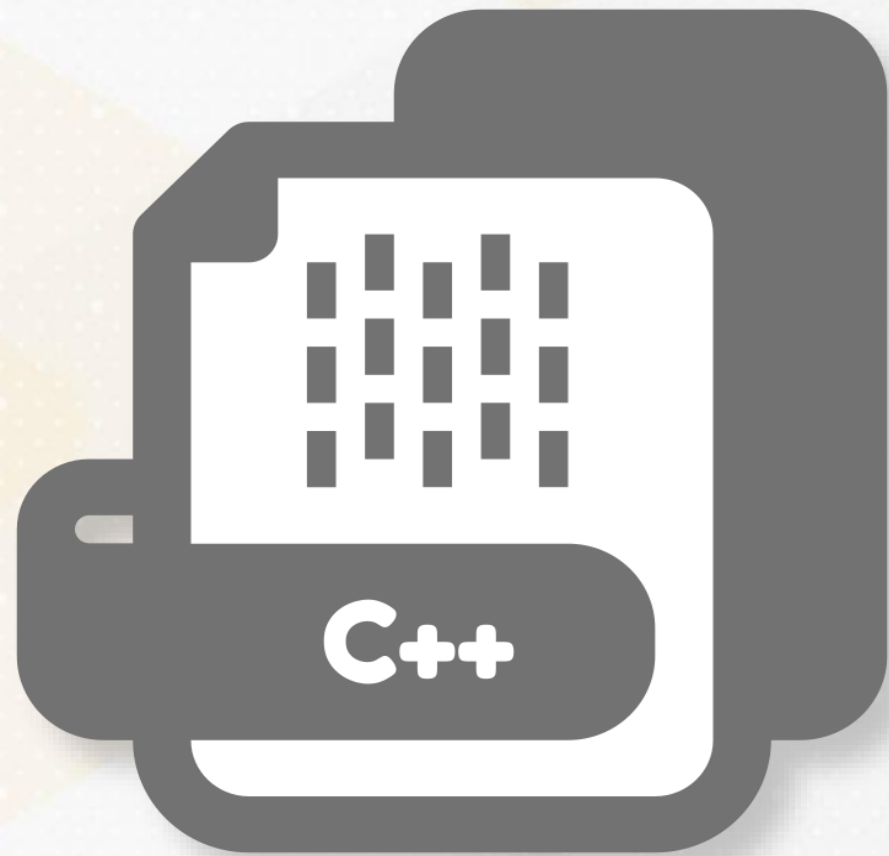
BuzzDB



BuzzDB



BuzzDB



Tuple Class

```
class Tuple {  
public:  
    int key; // Identifier column (e.g., customer id)  
    int value; // Data associated with key (e.g., transaction amount)  
};
```

Tuple Class

Tuple =
Row =
Record

```
class Tuple {  
public:  
    int key; // Identifier column (e.g., customer id)  
    int value; // Data associated with key (e.g., transaction amount)  
};
```

Tuple Class

Tuple =
Row =
Record

Key
Column
(Identifier)

```
class Tuple {  
public:  
    int key; // Identifier column (e.g., customer id)  
    int value; // Data associated with key (e.g., transaction amount)  
};
```


Tuple Class

Tuple =
Row =
Record

Key
Column
(Identifier)

Value
Column
(Data)

```
class Tuple {  
public:  
    int key; // Identifier column (e.g., customer id)  
    int value; // Data associated with key (e.g., transaction amount)  
};
```

Classes in C++



```
class Book {  
    public :  
        std::string title;  
        std::string author;  
        void displayInfo() {  
            std::cout << "Title: " << title << "\nAuthor: " << author  
                << std::endl;  
        }  
};
```

Classes in C++



Book Class

```
class Book {  
    public :  
        std::string title;  
        std::string author;  
        void displayInfo() {  
            std::cout << "Title: " << title << "\nAuthor: " << author  
                << std::endl;  
        }  
};
```


Classes in C++



Book Class

displayInfo

```
class Book {  
    public :  
        std::string title;  
        std::string author;  
        void displayInfo() {  
            std::cout << "Title: " << title << "\nAuthor: " << author  
                << std::endl;  
        }  
};
```

Classes in C++



Book Class

displayInfo

Encapsulation

```
class Book {  
    public :  
    std::string title;  
    std::string author;  
    void displayInfo() {  
        std::cout << "Title: " << title << "\nAuthor: " << author  
            << std::endl;  
    }  
};
```

Classes in C++

```
Book myBook; // Create or instantiate a Book object  
myBook.title = "1984"; // Set book title  
myBook.author = "George Orwell"; // Set book author  
myBook.displayInfo(); // Display book information
```


Classes in C++

Book
“Object”

```
Book myBook; // Create or instantiate a Book object  
myBook.title = "1984"; // Set book title  
myBook.author = "George Orwell"; // Set book author  
myBook.displayInfo(); // Display book information
```

Classes in C++

Book
“Object”

Set
Attributes

```
Book myBook; // Create or instantiate a Book object  
myBook.title = "1984"; // Set book title  
myBook.author = "George Orwell"; // Set book author  
myBook.displayInfo(); // Display book information
```

Classes in C++

Book
“Object”

Set
Attributes

Methods =
Actions

```
Book myBook; // Create or instantiate a Book object  
myBook.title = "1984"; // Set book title  
myBook.author = "George Orwell"; // Set book author  
myBook.displayInfo(); // Display book information
```


Classes in C++

Classes in C++

Organize
Data and
Functions

Classes in C++

Organize
Data and
Functions

Modular
Code

Classes in C++

Organize
Data and
Functions

Modular
Code

Create
Complex
Data Types

Classes in C++: Encapsulation



Classes in C++: Encapsulation



Attribute Protection

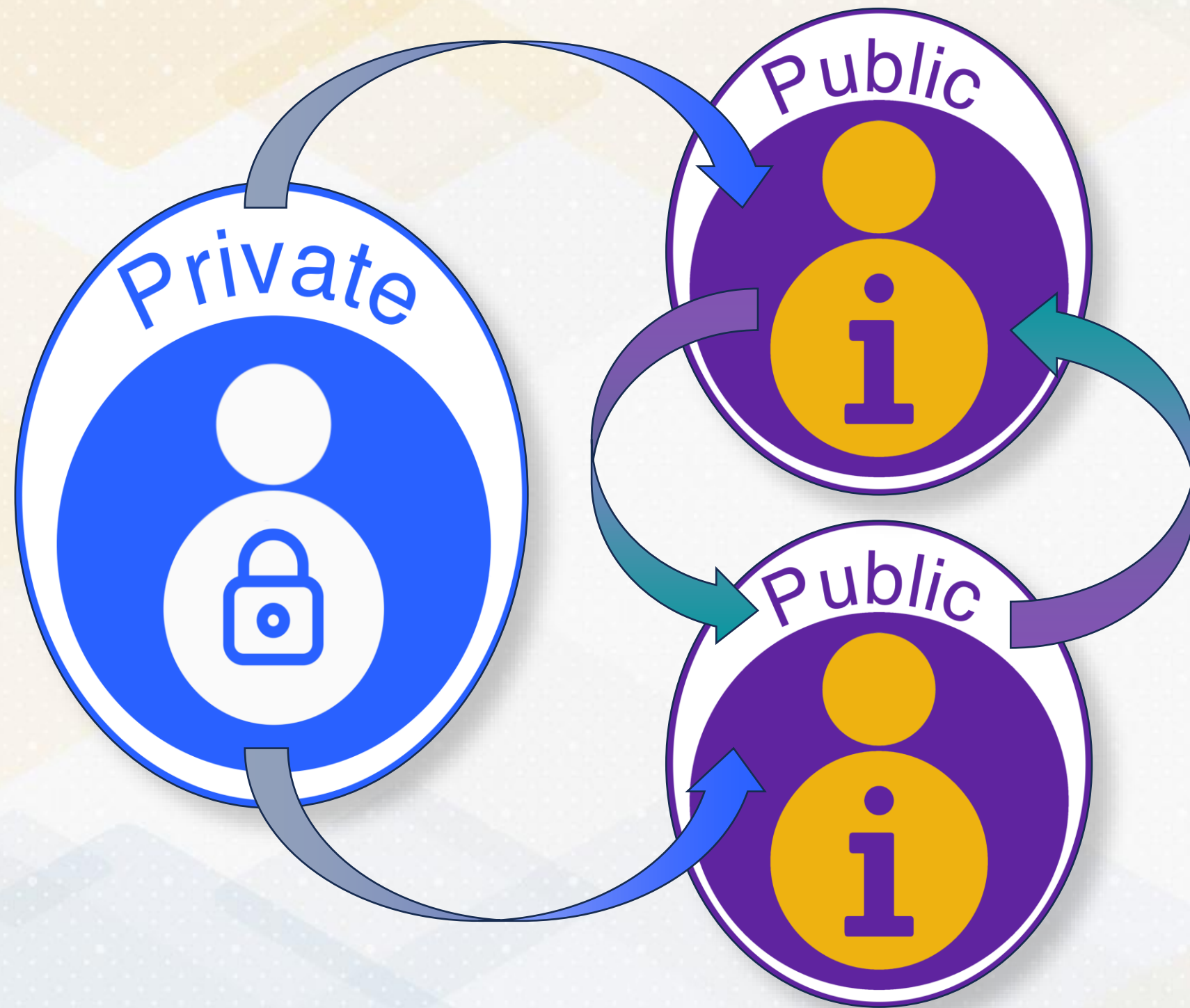
Classes in C++: Encapsulation



Attribute Protection

Public & Private

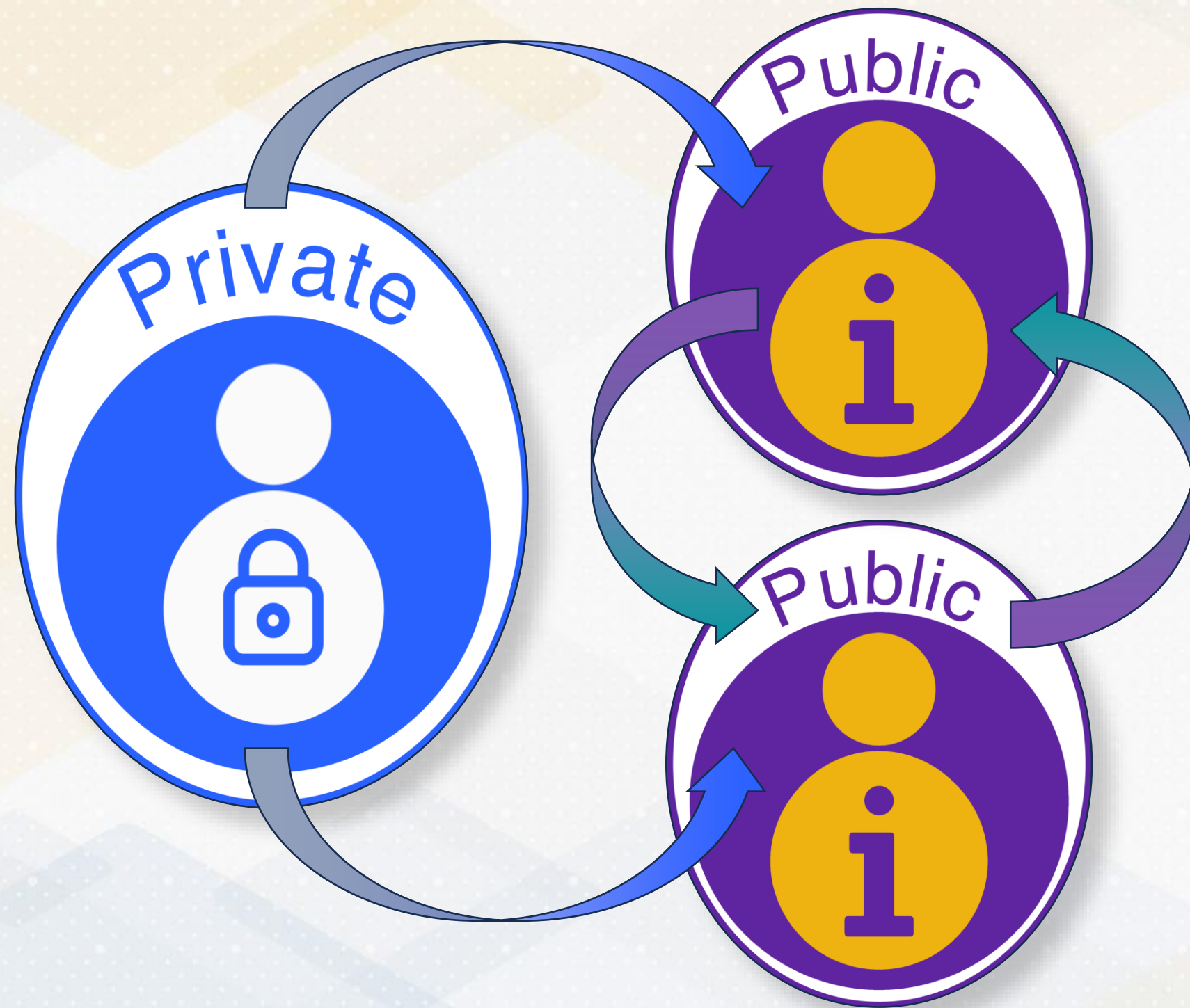
Classes in C++: Encapsulation



Attribute Protection

Public & Private

Classes in C++: Encapsulation



Attribute Protection

Public & Private

Protected Access

Classes in C++: Encapsulation



Classes in C++: Encapsulation



Storage Manager Class

Buffer Manager Class

B+Tree Index Class



BuzzDB Class

```
class BuzzDB {  
    public :  
        // Stores all our Tuples  
        std::vector<Tuple> table;  
  
    private :  
        // Index to quickly find the data associated with a key  
        std::map<int, std::vector<int>> index;  
};
```

BuzzDB Class

Vector for Storing Tuples

Map for Indexing Tuples

```
class BuzzDB {  
    public :  
        // Stores all our Tuples  
        std::vector<Tuple> table;  
  
    private :  
        // Index to quickly find the data associated with a key  
        std::map<int, std::vector<int>> index;  
};
```


C++ Containers

Vectors in C++



```
#include <vector>

// Declare a vector of integers
std::vector<int> myVector;
// Add an element to the end
myVector.push_back(10);
// Add another element
myVector.push_back(20);

// Access the first element (10)
std::cout << myVector[0];
```


Vectors in C++



C++ Standard Library

```
#include <vector>

// Declare a vector of integers
std::vector<int> myVector;
// Add an element to the end
myVector.push_back(10);
// Add another element
myVector.push_back(20);

// Access the first element (10)
std::cout << myVector[0];
```

Vectors in C++



C++ Standard
Library

std::vector

```
#include <vector>
```

```
// Declare a vector of integers
```

```
std::vector<int> myVector;
```

```
// Add an element to the end
```

```
myVector.push_back(10);
```

```
// Add another element
```

```
myVector.push_back(20);
```

```
// Access the first element (10)
```

```
std::cout << myVector[0];
```


Vectors in C++



C++ Standard
Library

std::vector

Dynamic
Size

```
#include <vector>

// Declare a vector of integers
std::vector<int> myVector;
// Add an element to the end
myVector.push_back(10);
// Add another element
myVector.push_back(20);

// Access the first element (10)
std::cout << myVector[0];
```

Vectors in C++

```
// Declare a vector for Tuple objects
std::vector<Tuple> tuples;
// Adding Tuple objects to our vector
tuples.push_back(Tuple(1, 100));
tuples.push_back(Tuple(2, 200));

// Accessing and displaying elements of the vector
for (const auto &tuple : tuples) {
    std::cout << "Key: " << tuple.key << ", Value: " << tuple.value
        << std::endl;
}
```


Vectors in C++

Versatile

```
// Declare a vector for Tuple objects
std::vector<Tuple> tuples;
// Adding Tuple objects to our vector
tuples.push_back(Tuple(1, 100));
tuples.push_back(Tuple(2, 200));

// Accessing and displaying elements of the vector
for (const auto &tuple : tuples) {
    std::cout << "Key: " << tuple.key << ", Value: " << tuple.value
        << std::endl;
}
```

Vectors in C++

Versatile

Any Type

```
// Declare a vector for Tuple objects
std::vector<Tuple> tuples;
// Adding Tuple objects to our vector
tuples.push_back(Tuple(1, 100));
tuples.push_back(Tuple(2, 200));

// Accessing and displaying elements of the vector
for (const auto &tuple : tuples) {
    std::cout << "Key: " << tuple.key << ", Value: " << tuple.value
        << std::endl;
}
```

Vectors in C++

Versatile

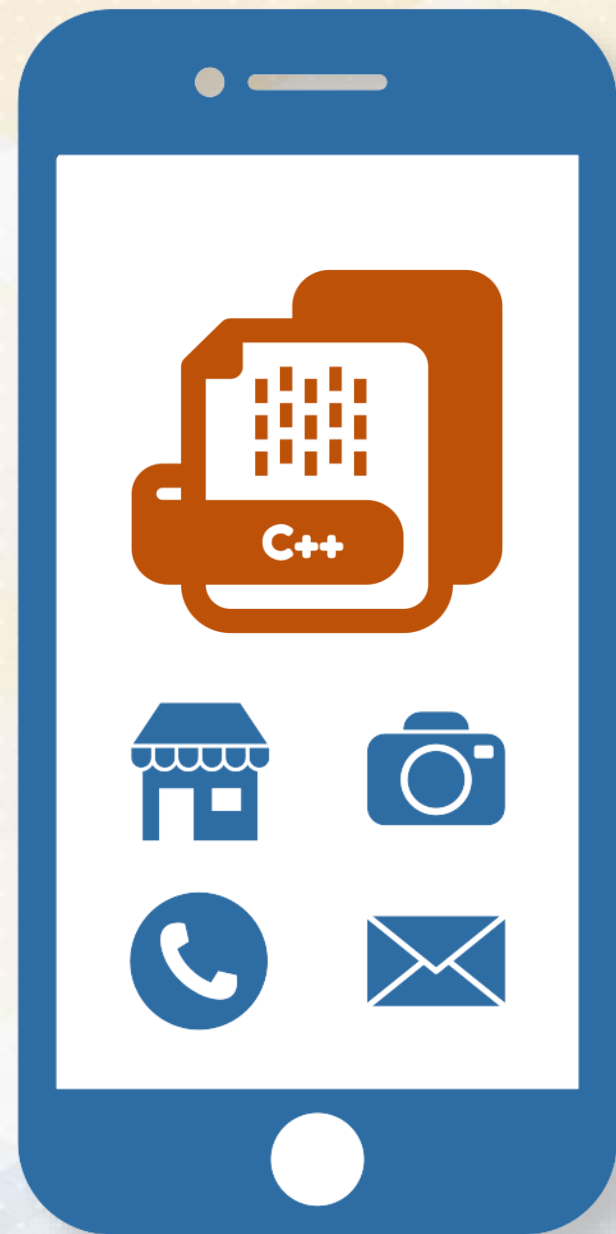
Any Type

Store Tuples

```
// Declare a vector for Tuple objects
std::vector<Tuple> tuples;
// Adding Tuple objects to our vector
tuples.push_back(Tuple(1, 100));
tuples.push_back(Tuple(2, 200));

// Accessing and displaying elements of the vector
for (const auto &tuple : tuples) {
    std::cout << "Key: " << tuple.key << ", Value: " << tuple.value
        << std::endl;
}
```


Maps in C++



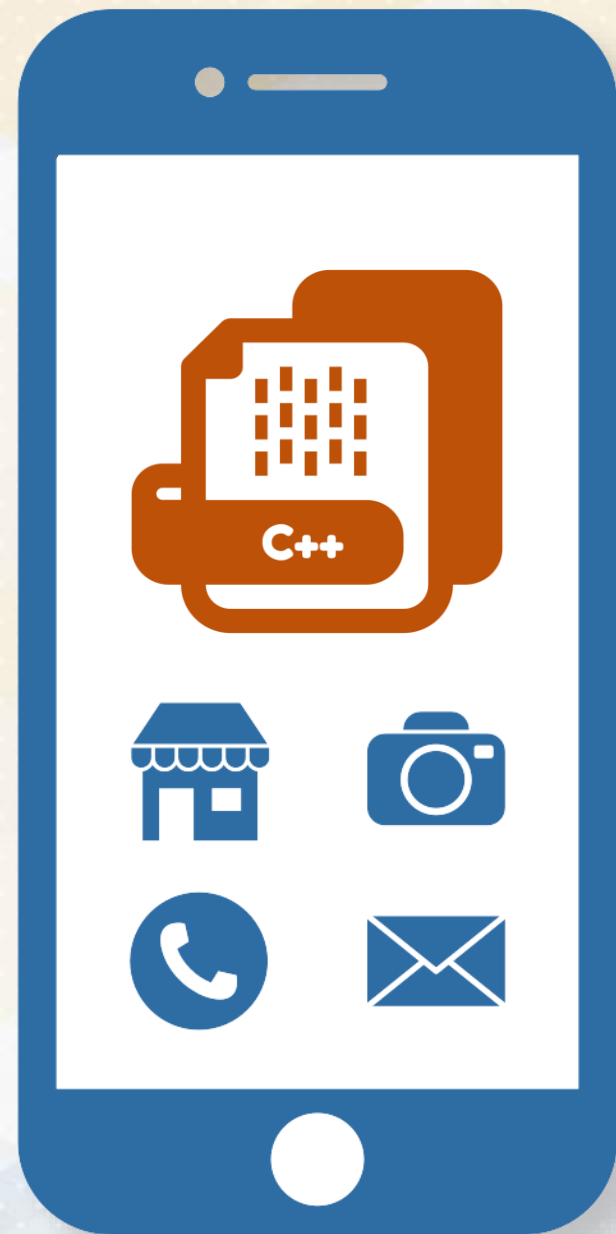
```
#include <map>
```

```
// Declare a map with string keys and int values  
std::map<std::string, int> myMap;
```

```
myMap["apple"] = 5; // Assign 5 to key "apple"  
myMap["banana"] = 10; // Assign 10 to key "banana"
```

```
// Access value associated with "apple" which will be 5  
std::cout << myMap["apple"];
```

Maps in C++



Associative
Container

Associates
Keys to Values

```
#include <map>
```

```
// Declare a map with string keys and int values  
std::map<std::string, int> myMap;
```

```
myMap["apple"] = 5; // Assign 5 to key "apple"  
myMap["banana"] = 10; // Assign 10 to key "banana"
```

```
// Access value associated with "apple" which will be 5  
std::cout << myMap["apple"];
```

Maps in C++



```
// Declaring a map with an int key and vector<int> as value
std::map<int, std::vector<int>> index;

// Adding data to the map
index[1].push_back(100);
index[1].push_back(200); // Multiple values for the same key 1
index[2].push_back(50);
```


Maps in C++



Keys kept in
Sorted order

Associate Multiple
Values to a Key

```
// Declaring a map with an int key and vector<int> as value
std::map<int, std::vector<int>> index;

// Adding data to the map
index[1].push_back(100);
index[1].push_back(200); // Multiple values for the same key 1
index[2].push_back(50);
```

Maps in C++



Keys kept in
Sorted order

Associate Multiple
Values to a Key

Map's
Key Type = *int*

Map's Value Type =
Vector of int

```
// Declaring a map with an int key and vector<int> as value
std::map<int, std::vector<int>> index;

// Adding data to the map
index[1].push_back(100);
index[1].push_back(200); // Multiple values for the same key 1
index[2].push_back(50);
```

Maps in C++



```
for (const auto &pair : index) {  
    std::cout << "Key: " << pair.first  
                << ", Values: ";  
    for (int value : pair.second) {  
        std::cout << value << ", ";  
    }  
    std::cout << std::endl;  
}
```

// PROGRAM OUTPUT

Key: 1, Values: 100, 200

Key: 2, Values: 50



BuzzDB Functions



Insertion Function in BuzzDB

Key	Value
1	100

```
void BuzzDB::insert(int key, int value) {  
    Tuple newTuple = {key, value};  
    // Add tuple to the table vector  
    table.push_back(newTuple);  
    // Also, add it to the index map  
    index[key].push_back(value);  
}
```

Insertion Function in BuzzDB

Key	Value
1	100
1	200
2	50
3	200
3	200

```
void BuzzDB::insert(int key, int value) {  
    Tuple newTuple = {key, value};  
    // Add tuple to the table vector  
    table.push_back(newTuple);  
    // Also, add it to the index map  
    index[key].push_back(value);  
}
```

Add Tuple to Table Vector and Index Map

Populating the Database

```
int main() {  
    BuzzDB db;  
    // Populating the database  
    db.insert(1, 100);  
    db.insert(1, 200);  
    db.insert(2, 50);  
    db.insert(3, 200);  
    db.insert(3, 200);  
    db.insert(3, 100);  
    db.insert(4, 500);  
    // Executing aggregation query  
    db.selectGroupBySum();  
    return 0;  
}
```

Aggregation Query in BuzzDB

```
void BuzzDB::selectGroupBySum() {  
    // Iterate over each key  
    for (auto const &pair : index) {  
        int sum = 0;  
        // Sum values for this key  
        for (auto const &value : pair.second) {  
            sum += value;  
        }  
        std::cout << "key: " << pair.first  
                    << ", sum: " << sum << '\n';  
    }  
}
```

Aggregation Query in BuzzDB

selectGroupBySum method

```
void BuzzDB::selectGroupBySum() {  
    // Iterate over each key  
    for (auto const &pair : index) {  
        int sum = 0;  
        // Sum values for this key  
        for (auto const &value : pair.second) {  
            sum += value;  
        }  
        std::cout << "key: " << pair.first  
                    << ", sum: " << sum << '\n';  
    }  
}
```


Aggregation Query in BuzzDB

selectGroupBySum method

Sum values based on keys

```
void BuzzDB::selectGroupBySum() {  
    // Iterate over each key  
    for (auto const &pair : index) {  
        int sum = 0;  
        // Sum values for this key  
        for (auto const &value : pair.second) {  
            sum += value;  
        }  
        std::cout << "key: " << pair.first  
                    << ", sum: " << sum << '\n';  
    }  
}
```

Aggregation Query in BuzzDB

selectGroupBySum method

Sum values based on keys

Iterate over Keys

```
void BuzzDB::selectGroupBySum() {  
    // Iterate over each key  
    for (auto const &pair : index) {  
        int sum = 0;  
        // Sum values for this key  
        for (auto const &value : pair.second) {  
            sum += value;  
        }  
        std::cout << "key: " << pair.first  
                    << ", sum: " << sum << '\n';  
    }  
}
```

Aggregation Query in BuzzDB

selectGroupBySum method

Sum values based on keys

Iterate over Keys

Iterate over Value Vector

```
void BuzzDB::selectGroupBySum() {  
    // Iterate over each key  
    for (auto const &pair : index) {  
        int sum = 0;  
        // Sum values for this key  
        for (auto const &value : pair.second) {  
            sum += value;  
        }  
        std::cout << "key: " << pair.first  
            << ", sum: " << sum << '\n';  
    }  
}
```


Aggregation Query

```
int main() {  
    ...  
    // Executing aggregation query  
    db.selectGroupBySum();  
    return 0;  
}
```

// PROGRAM OUTPUT

key: 1, sum: 300

key: 2, sum: 50

key: 3, sum: 500

key: 4, sum: 500

Aggregation Query

Database
Loading

Aggregate
Query

```
int main() {  
    ...  
    // Executing aggregation query  
    db.selectGroupBySum();  
    return 0;  
}
```

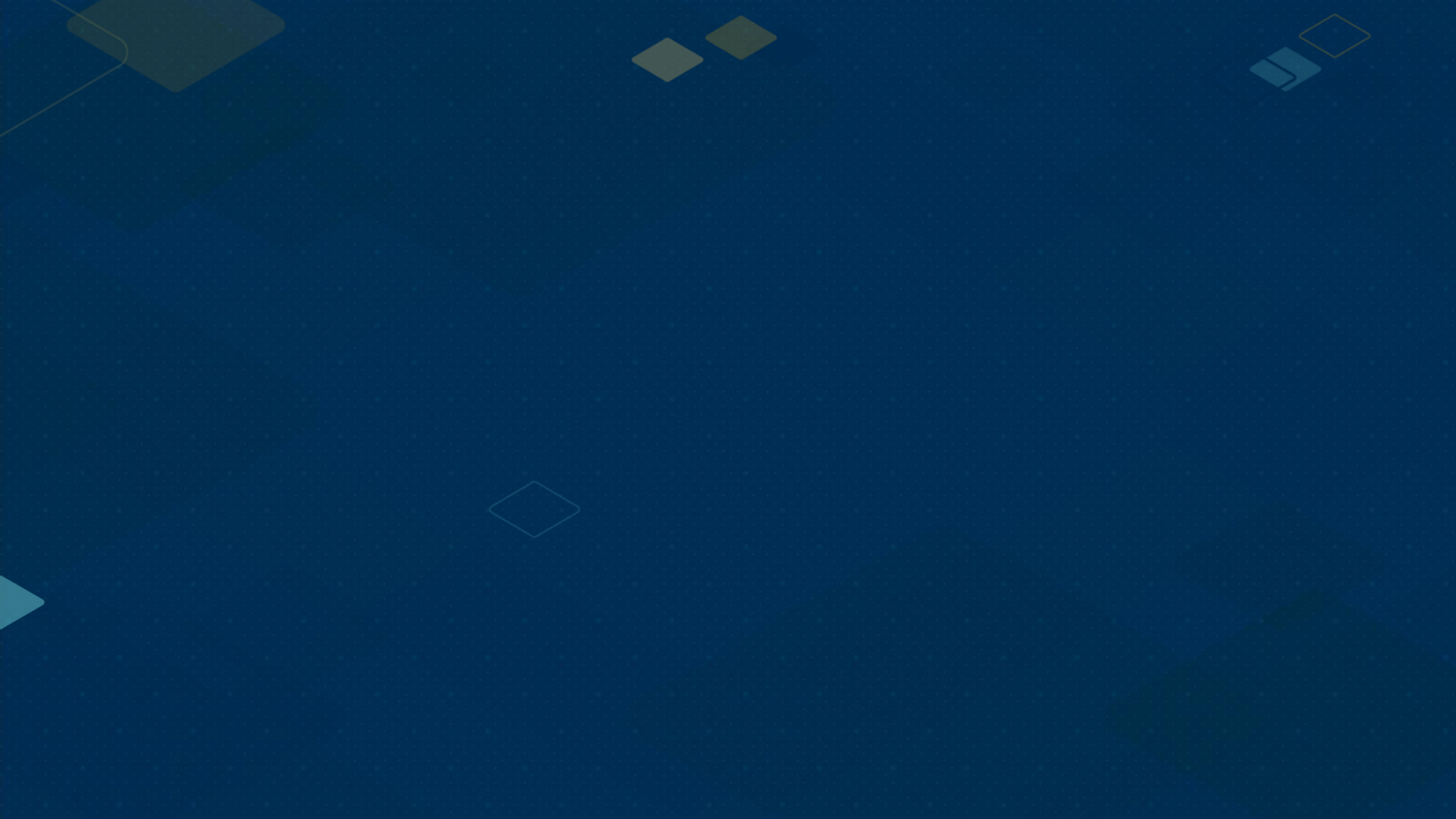
// PROGRAM OUTPUT

key: 1, sum: 300

key: 2, sum: 50

key: 3, sum: 500

key: 4, sum: 500





BuzzDB



BuzzDB

**Fundamentals
of C++**

Why C++?





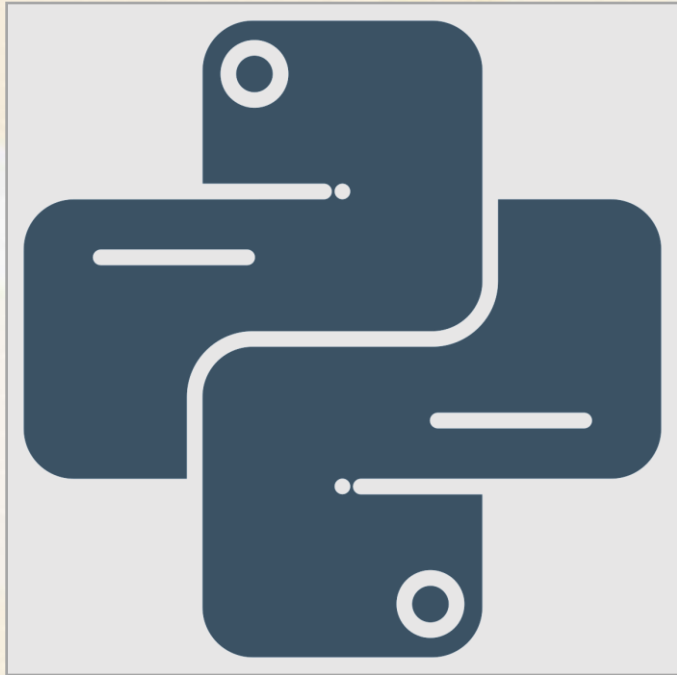
Why C++?



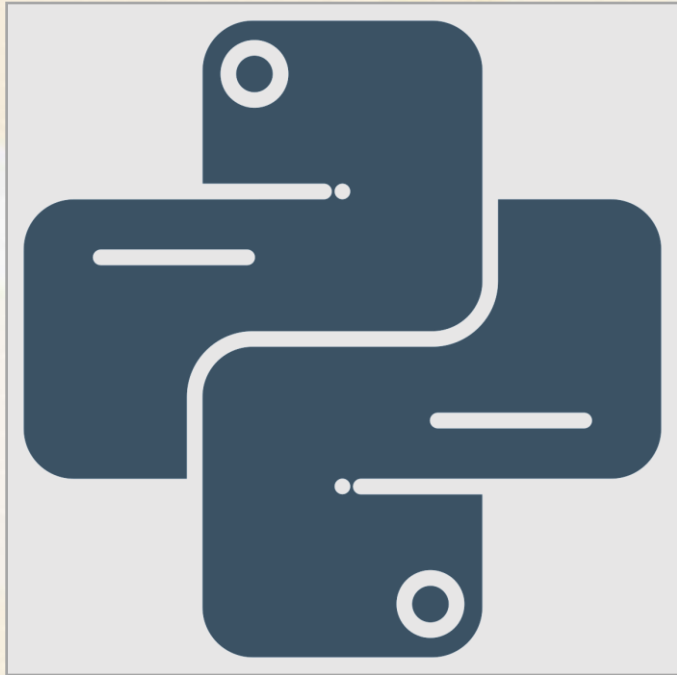
Why C++?

**Performance
and Hardware
Control**

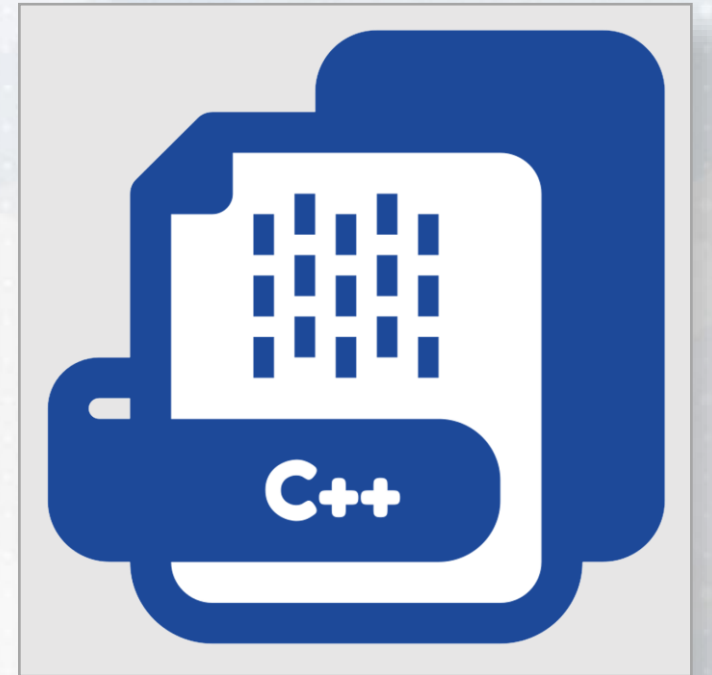
C++ vs Python



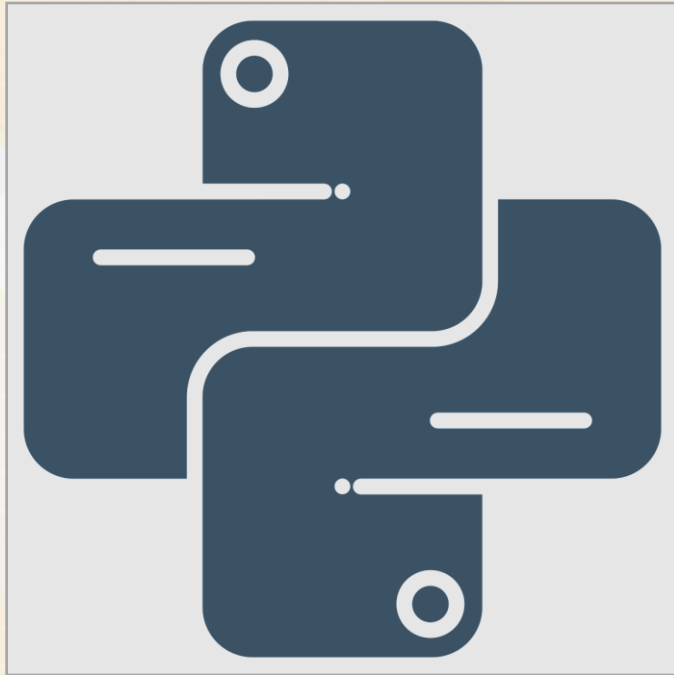
C++ vs Python



Superior Performance



C++ vs Python

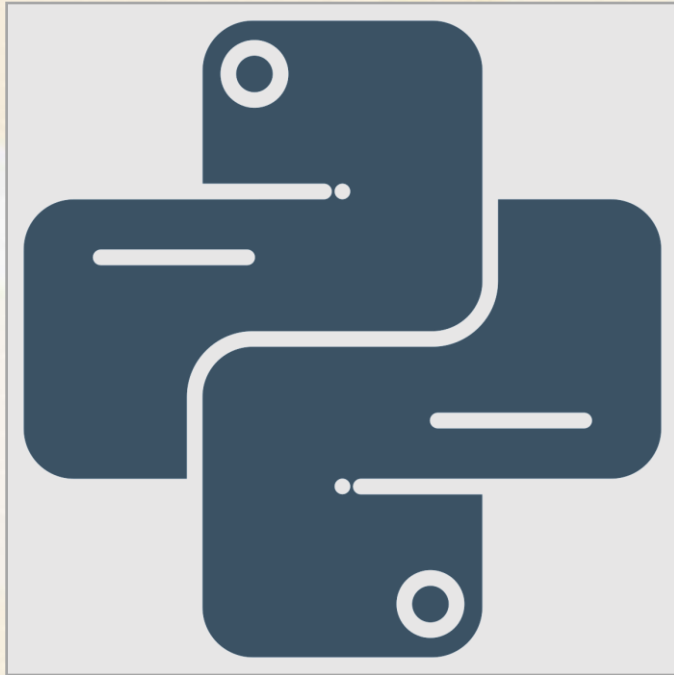


Superior Performance

Compiled Language



C++ vs Python



Superior Performance

Compiled Language

Fast and Efficient



Performance: C++

```
#include <chrono>
#include <iostream>
int main() {
    auto start = std::chrono::high_resolution_clock::now();
    long sum = 0;
    for (int i = 0; i < 1000000000; ++i) {
        sum += i;
    }
    auto end = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double> diff = end - start;
    std::cout << "Time taken: " << diff.count() << " seconds\n";
}
```

Performance: C++

Sum 100 M
integers

```
#include <chrono>
#include <iostream>
int main() {
    auto start = std::chrono::high_resolution_clock::now();
    long sum = 0;
    for (int i = 0; i < 1000000000; ++i) {
        sum += i;
    }
    auto end = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double> diff = end - start;
    std::cout << "Time taken: " << diff.count() << " seconds\n";
}
```


Performance: C++

Sum 100 M
integers

Chrono
Library

```
#include <chrono>
#include <iostream>
int main() {
    auto start = std::chrono::high_resolution_clock::now();
    long sum = 0;
    for (int i = 0; i < 1000000000; ++i) {
        sum += i;
    }
    auto end = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double> diff = end - start;
    std::cout << "Time taken: " << diff.count() << " seconds\n";
}
```

Performance: C++

Sum 100 M
integers

Chrono
Library

100
microseconds

```
#include <chrono>
#include <iostream>
int main() {
    auto start = std::chrono::high_resolution_clock::now();
    long sum = 0;
    for (int i = 0; i < 1000000000; ++i) {
        sum += i;
    }
    auto end = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double> diff = end - start;
    std::cout << "Time taken: " << diff.count() << " seconds\n";
}
```

Performance: Python

```
import time

start = time.time()

sum = 0
for i in range(100000000):
    sum += i

end = time.time()
print(f"Time taken: {end - start} seconds")
```


Performance: Python

5.24
seconds

```
import time

start = time.time()

sum = 0
for i in range(100000000):
    sum += i

end = time.time()
print(f"Time taken: {end - start} seconds")
```

Performance: Python

5.24
seconds

50000 times
slower

```
import time

start = time.time()

sum = 0
for i in range(100000000):
    sum += i

end = time.time()
print(f"Time taken: {end - start} seconds")
```

Performance: Python

5.24
seconds

50000 times
slower

Interpreted
Language

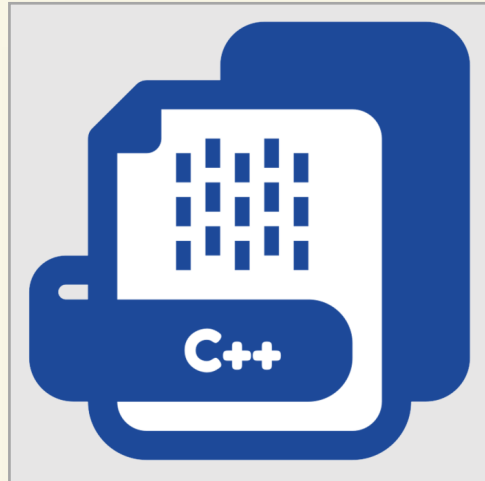
```
import time

start = time.time()

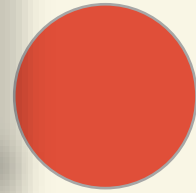
sum = 0
for i in range(100000000):
    sum += i

end = time.time()
print(f"Time taken: {end - start} seconds")
```

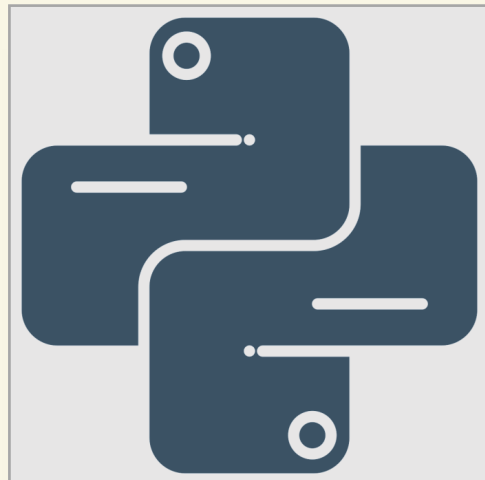
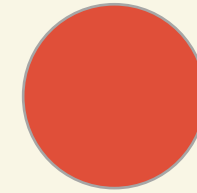

Performance Comparison



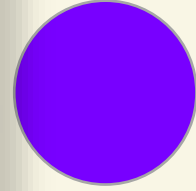
Atlanta



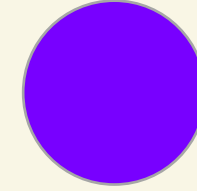
New York



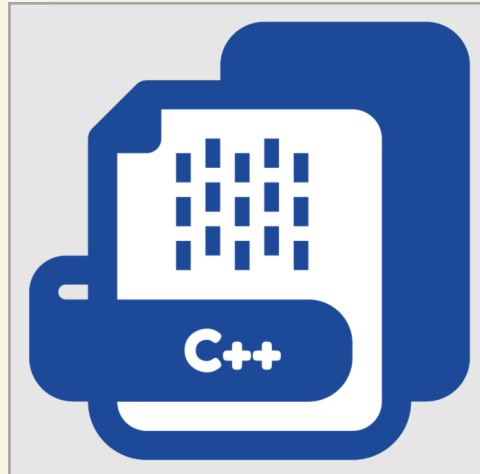
Earth



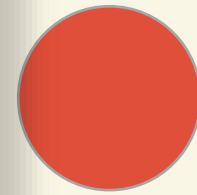
Pluto



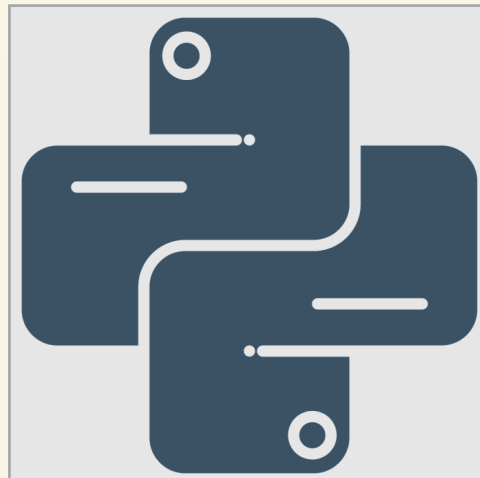
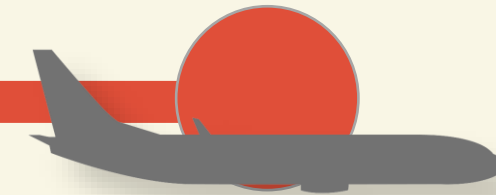
Performance Comparison



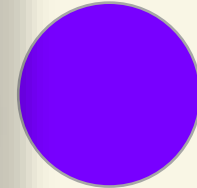
Atlanta



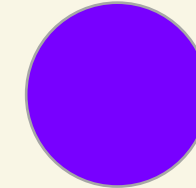
New York



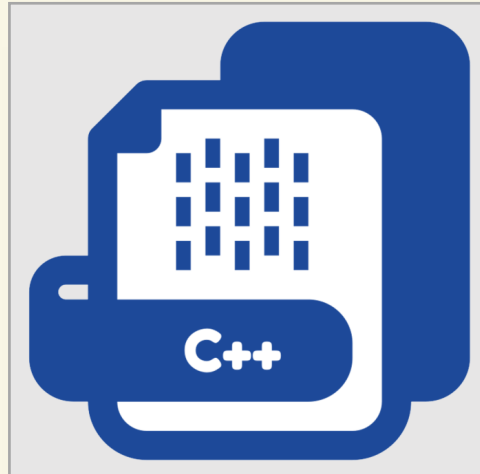
Earth



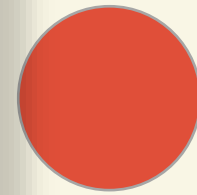
Pluto



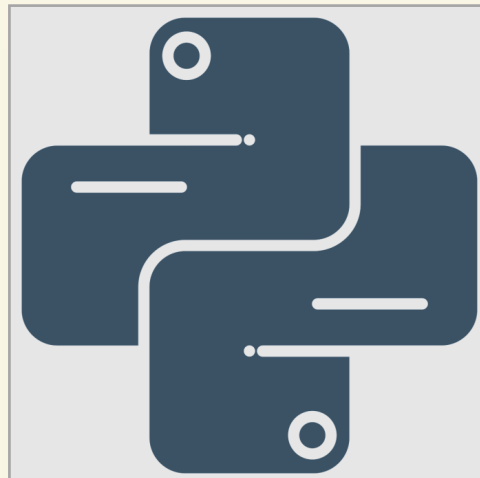
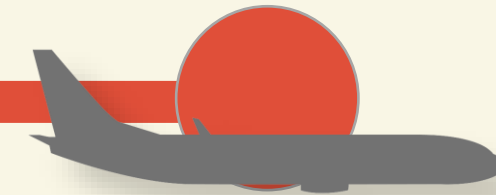
Performance Comparison



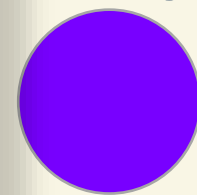
Atlanta



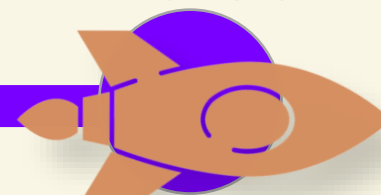
New York



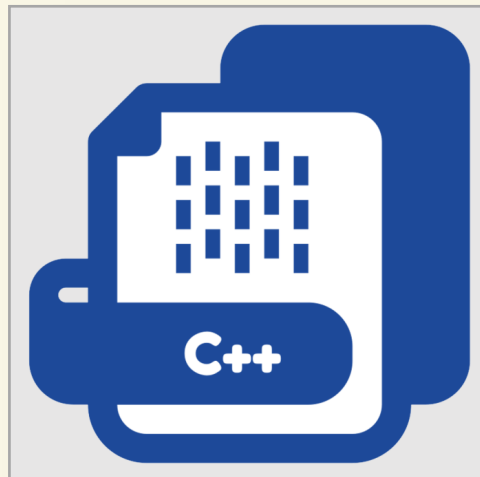
Earth



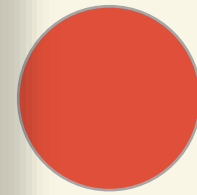
Pluto



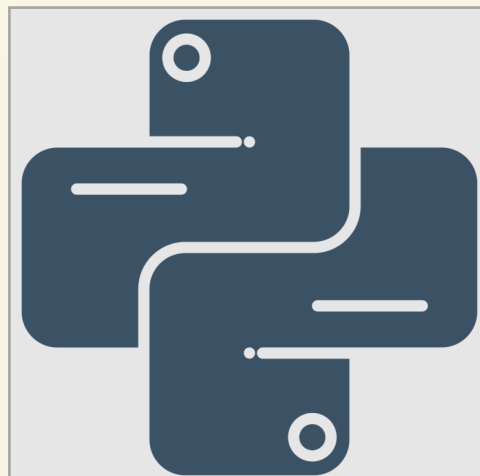
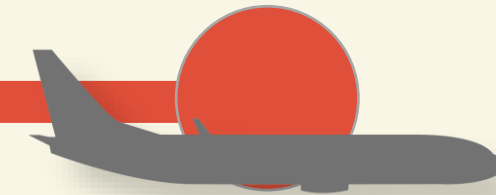
Performance Comparison



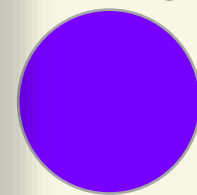
Atlanta



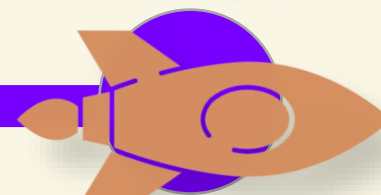
New York



Earth

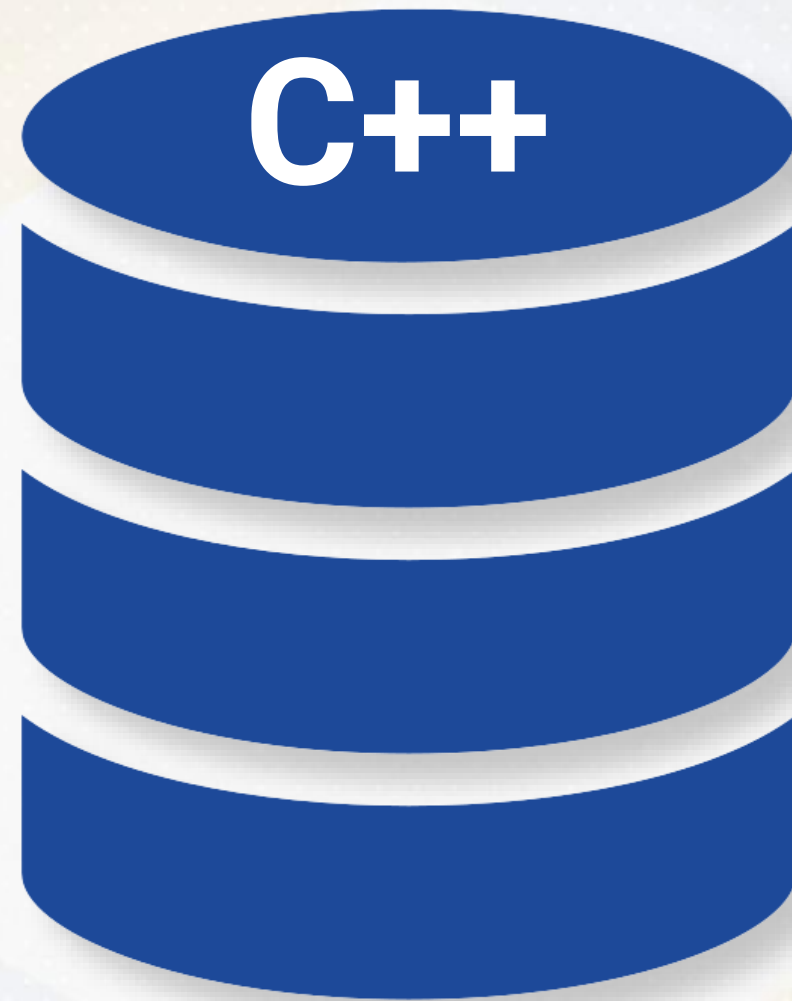


Pluto



Similar to gap between C++ and Python

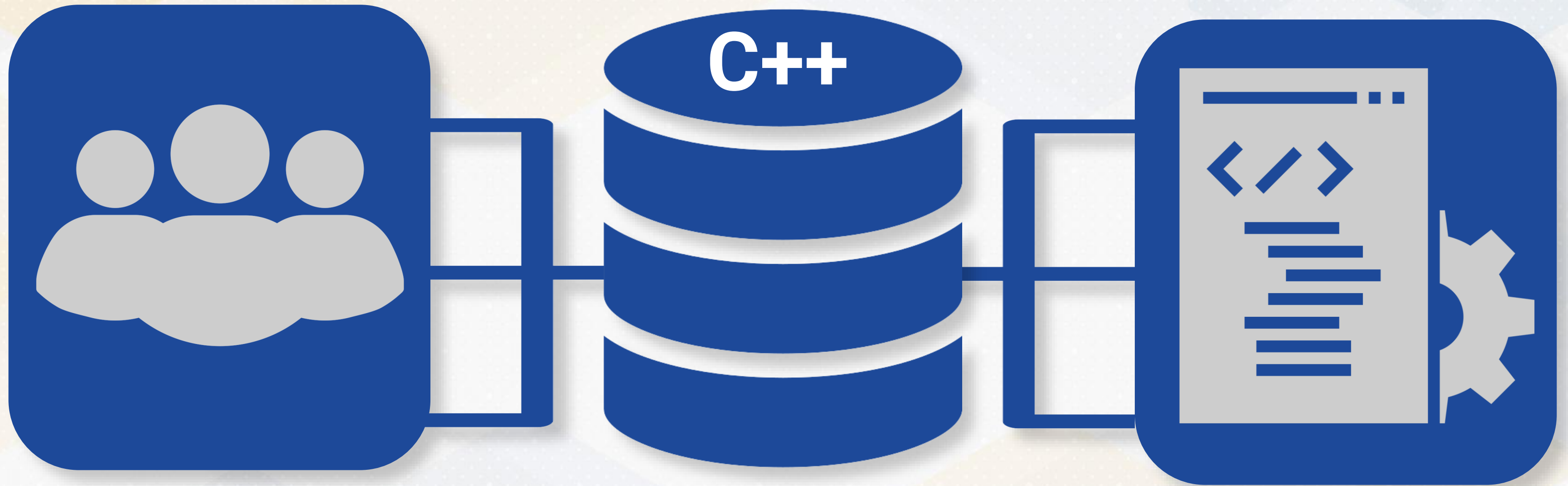
C++ vs Python: Hardware Control



C++ vs Python: Hardware Control



C++ vs Python: Hardware Control



C++: Memory Management

```
#include <memory>
std::unique_ptr<int[]> allocateArray(int size) {
    return std::make_unique<int[]>(size);
}
int main() {
    auto myArray = allocateArray(1000000);
    for (int i = 0; i < 1000000; ++i) {
        myArray[i] = i;
    }
    std::cout << "Array[500000] = "
              << myArray[500000] << std::endl;
    // No need to manually delete the array,
    // as memory is automatically freed when out of scope
}
```

C++: Memory Management

std::unique_ptr

```
#include <memory>
std::unique_ptr<int[]> allocateArray(int size) {
    return std::make_unique<int[]>(size);
}
int main() {
    auto myArray = allocateArray(1000000);
    for (int i = 0; i < 1000000; ++i) {
        myArray[i] = i;
    }
    std::cout << "Array[500000] = "
              << myArray[500000] << std::endl;
    // No need to manually delete the array,
    // as memory is automatically freed when out of scope
}
```


C++: Memory Management

`std::unique_ptr`

`allocateArray`

```
#include <memory>
std::unique_ptr<int[]> allocateArray(int size) {
    return std::make_unique<int[]>(size);
}
int main() {
    auto myArray = allocateArray(1000000);
    for (int i = 0; i < 1000000; ++i) {
        myArray[i] = i;
    }
    std::cout << "Array[500000] = "
              << myArray[500000] << std::endl;
    // No need to manually delete the array,
    // as memory is automatically freed when out of scope
}
```

C++: Memory Management

`std::unique_ptr`

`allocateArray`

No memory leaks

```
#include <memory>
std::unique_ptr<int[]> allocateArray(int size) {
    return std::make_unique<int[]>(size);
}
int main() {
    auto myArray = allocateArray(1000000);
    for (int i = 0; i < 1000000; ++i) {
        myArray[i] = i;
    }
    std::cout << "Array[500000] = "
              << myArray[500000] << std::endl;
    // No need to manually delete the array,
    // as memory is automatically freed when out of scope
}
```

C++: Memory Management

`std::unique_ptr`

`allocateArray`

No memory leaks

`<memory>`

```
#include <memory>
std::unique_ptr<int[]> allocateArray(int size) {
    return std::make_unique<int[]>(size);
}
int main() {
    auto myArray = allocateArray(1000000);
    for (int i = 0; i < 1000000; ++i) {
        myArray[i] = i;
    }
    std::cout << "Array[500000] = "
              << myArray[500000] << std::endl;
    // No need to manually delete the array,
    // as memory is automatically freed when out of scope
}
```


C++: Multi-Threading

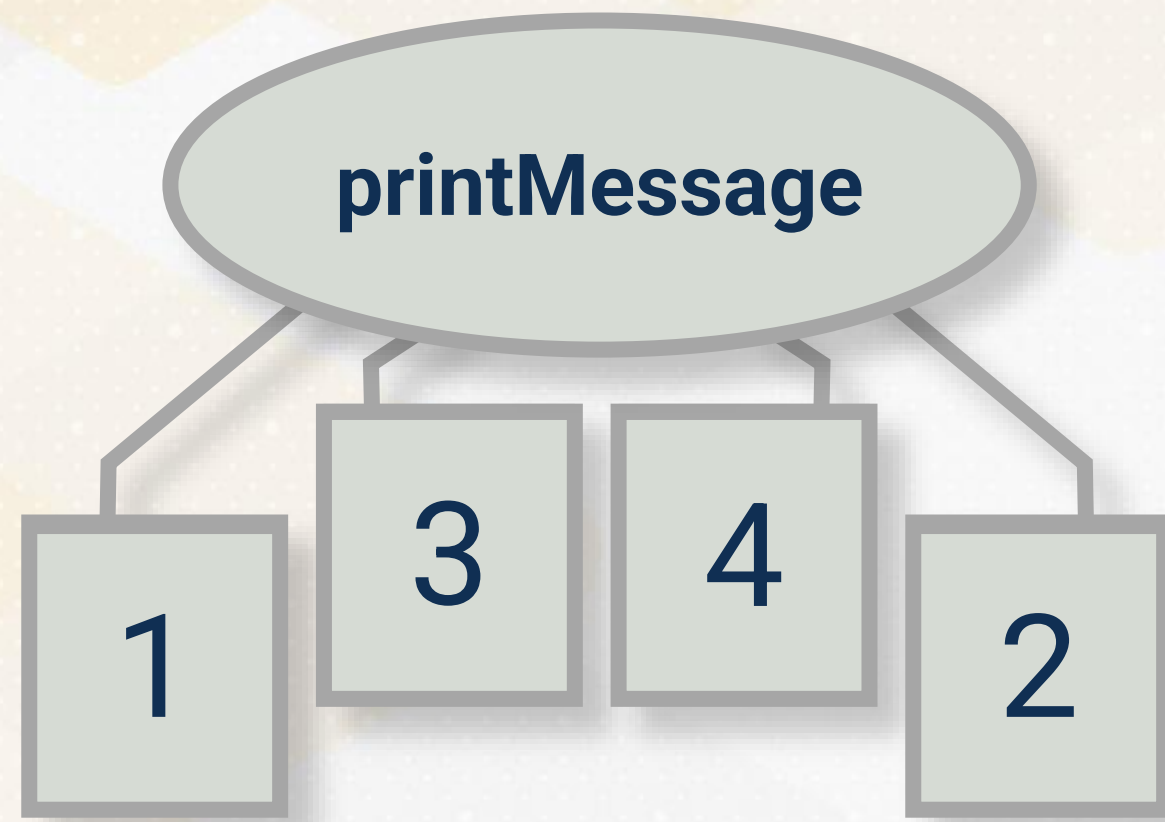
```
#include <thread>
// Function that each thread will execute
void printMessage(int threadId) {
    std::cout << "Hello from Thread " << threadId
               << std::endl;
}
int main() {
    const int numThreads = 4;
    std::vector<std::thread> threads;
    // Create and start threads
    for (int i = 0; i < numThreads; ++i) {
        threads.emplace_back(printMessage, i + 1);
    }
    // Wait for all threads to finish
    for (auto &t : threads) {
        t.join();
    }
}
```

C++: Multi-Threading

printMessage

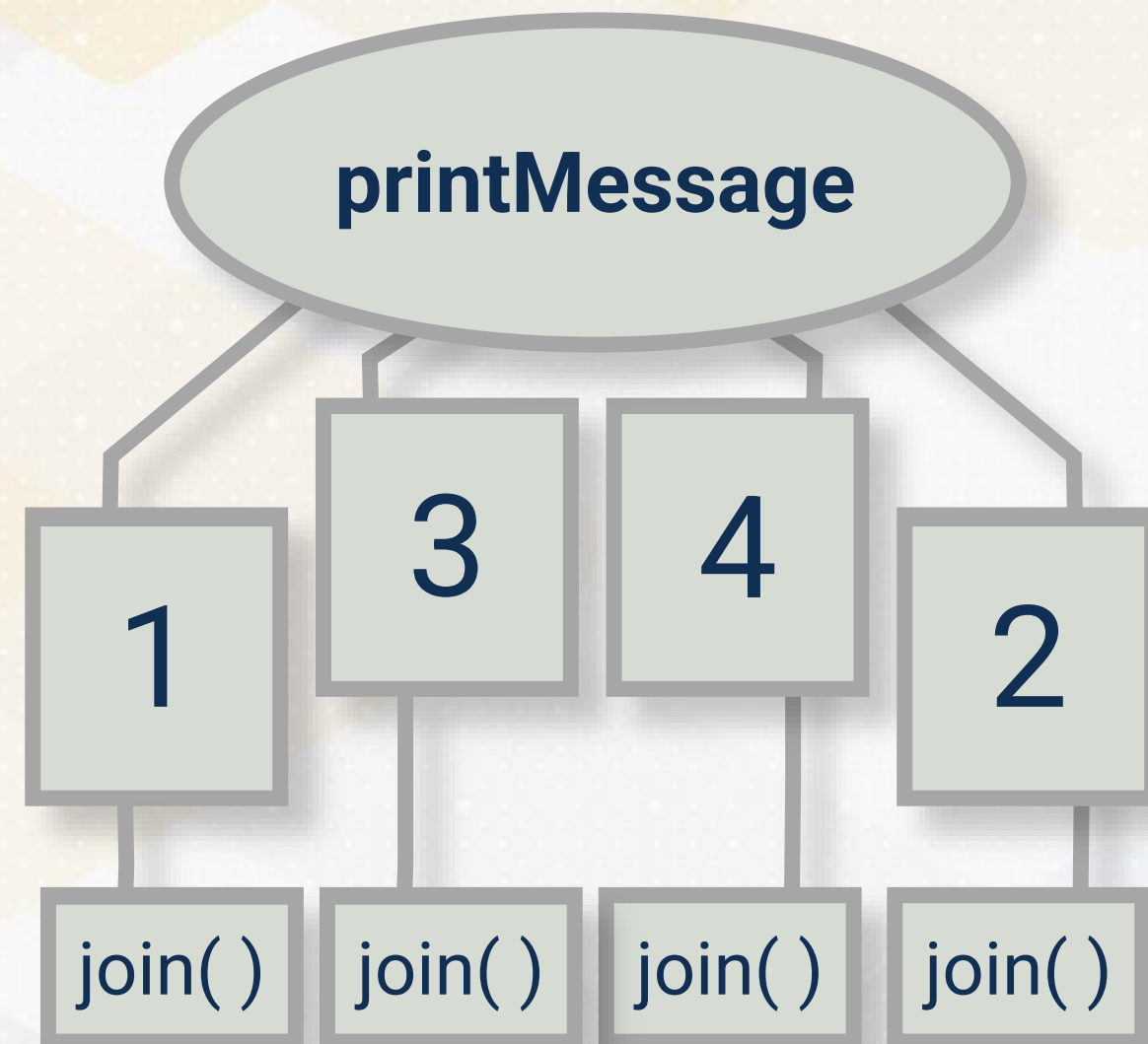
```
#include <thread>
// Function that each thread will execute
void printMessage(int threadId) {
    std::cout << "Hello from Thread " << threadId
               << std::endl;
}
int main() {
    const int numThreads = 4;
    std::vector<std::thread> threads;
    // Create and start threads
    for (int i = 0; i < numThreads; ++i) {
        threads.emplace_back(printMessage, i + 1);
    }
    // Wait for all threads to finish
    for (auto &t : threads) {
        t.join();
    }
}
```


C++: Multi-Threading



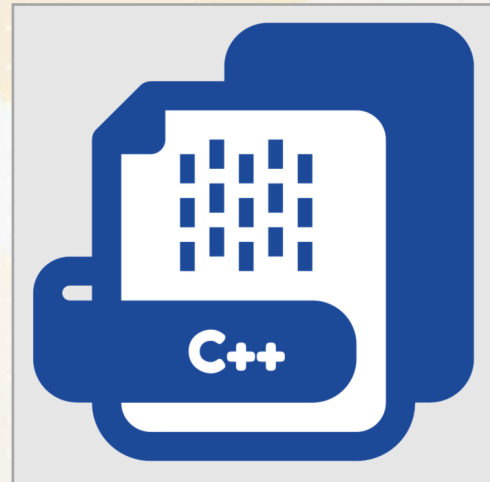
```
#include <thread>
// Function that each thread will execute
void printMessage(int threadId) {
    std::cout << "Hello from Thread " << threadId
               << std::endl;
}
int main() {
    const int numThreads = 4;
    std::vector<std::thread> threads;
    // Create and start threads
    for (int i = 0; i < numThreads; ++i) {
        threads.emplace_back(printMessage, i + 1);
    }
    // Wait for all threads to finish
    for (auto &t : threads) {
        t.join();
    }
}
```


C++: Multi-Threading

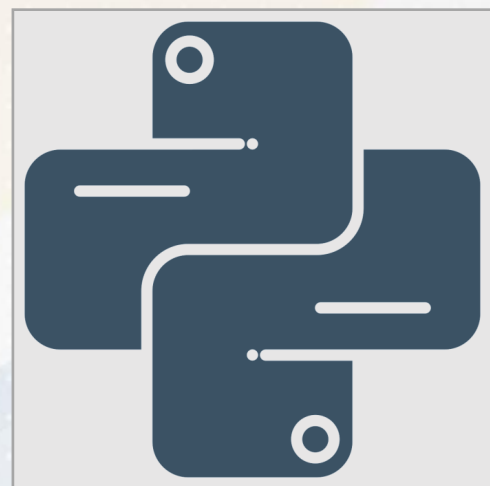


```
#include <thread>
// Function that each thread will execute
void printMessage(int threadId) {
    std::cout << "Hello from Thread " << threadId
               << std::endl;
}
int main() {
    const int numThreads = 4;
    std::vector<std::thread> threads;
    // Create and start threads
    for (int i = 0; i < numThreads; ++i) {
        threads.emplace_back(printMessage, i + 1);
    }
    // Wait for all threads to finish
    for (auto &t : threads) {
        t.join();
    }
}
```

C++ vs Python: Typing



```
int main() {  
    int myInt = 10; // Type must be specified  
    myInt = "Hello"; // Compile-time error  
}
```



```
# Initially an int  
my_var = 10  
# Can be reassigned to a string without error  
my_var = "Hello"
```

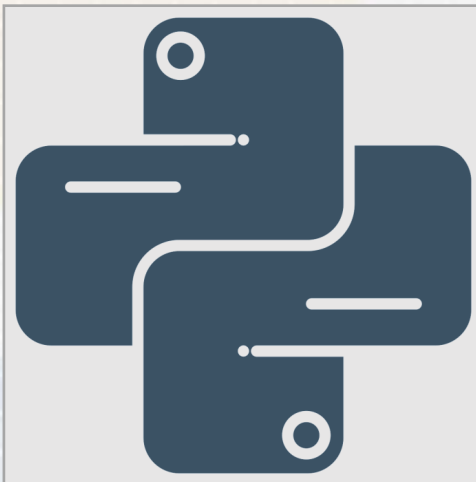
C++ vs Python: Typing



Compile-time Error

Performance

```
int main() {  
    int myInt = 10; // Type must be specified  
    myInt = "Hello"; // Compile-time error  
}
```



```
# Initially an int  
my_var = 10  
# Can be reassigned to a string without error  
my_var = "Hello"
```

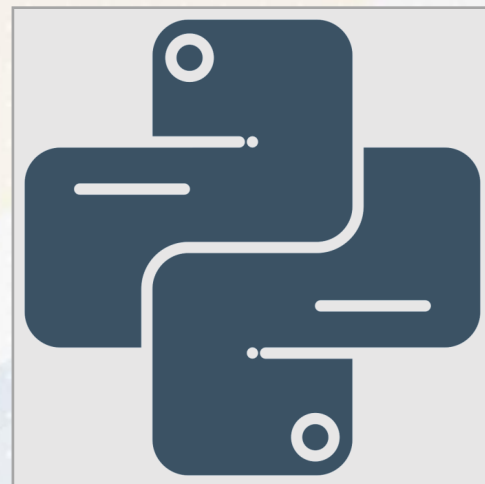

C++ vs Python: Typing



Compile-time Error

Performance

```
int main() {  
    int myInt = 10; // Type must be specified  
    myInt = "Hello"; // Compile-time error  
}
```



Run-time Error

Inefficient

```
# Initially an int  
my_var = 10  
# Can be reassigned to a string without error  
my_var = "Hello"
```





Why C++?

Why C++?

**Performance
and Hardware
Control**



Transactions







Transactions



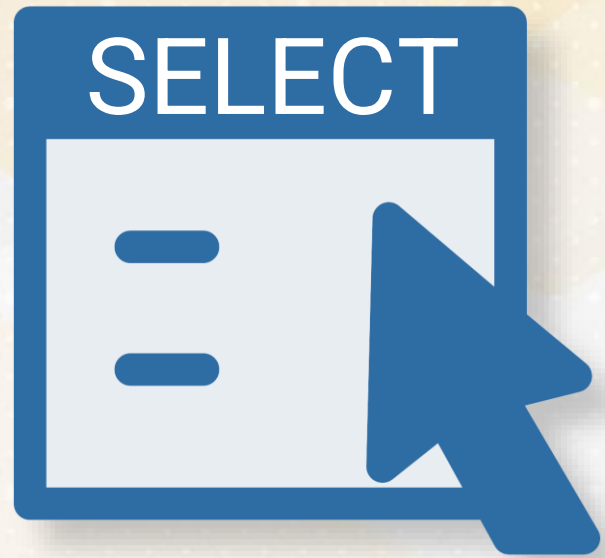
Transactions

**ACID
Properties**

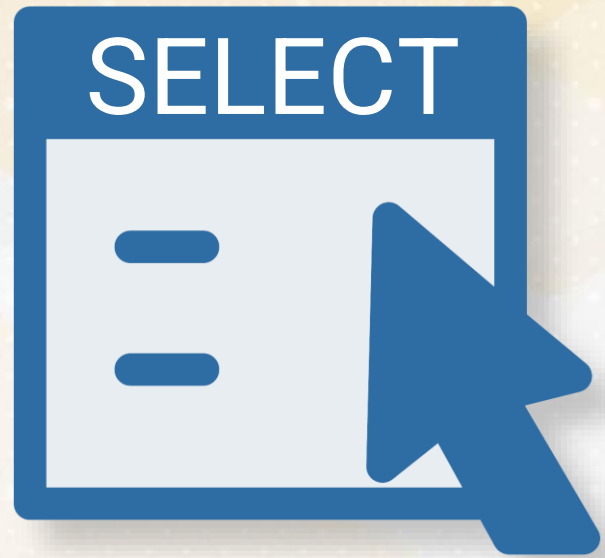
Queries → Transactions



Queries → Transactions

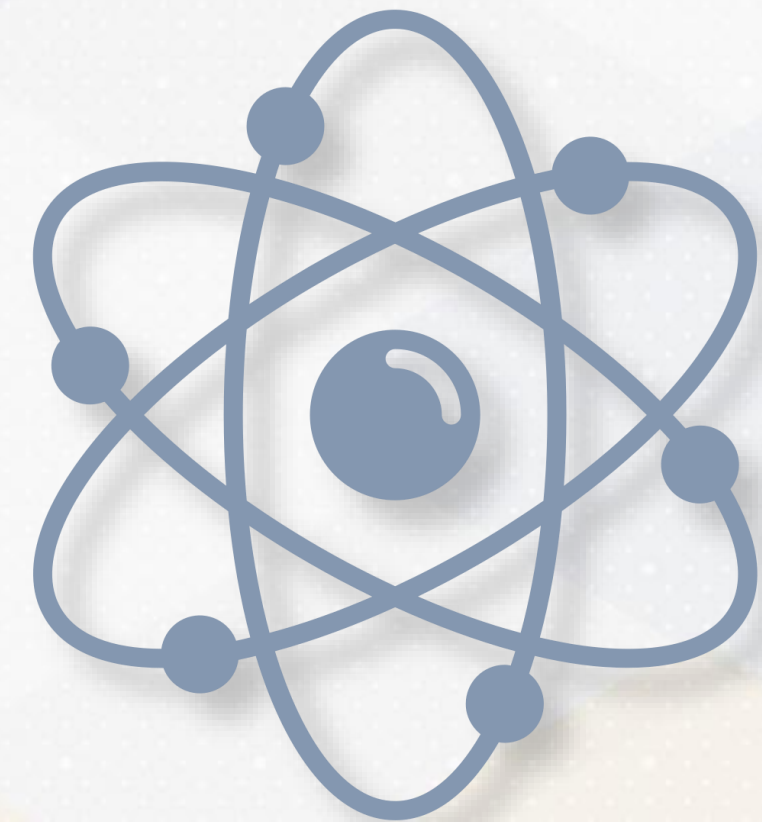


Queries → Transactions



✓ **Multiple Changes to Database**

Transactions: Atomicity Property



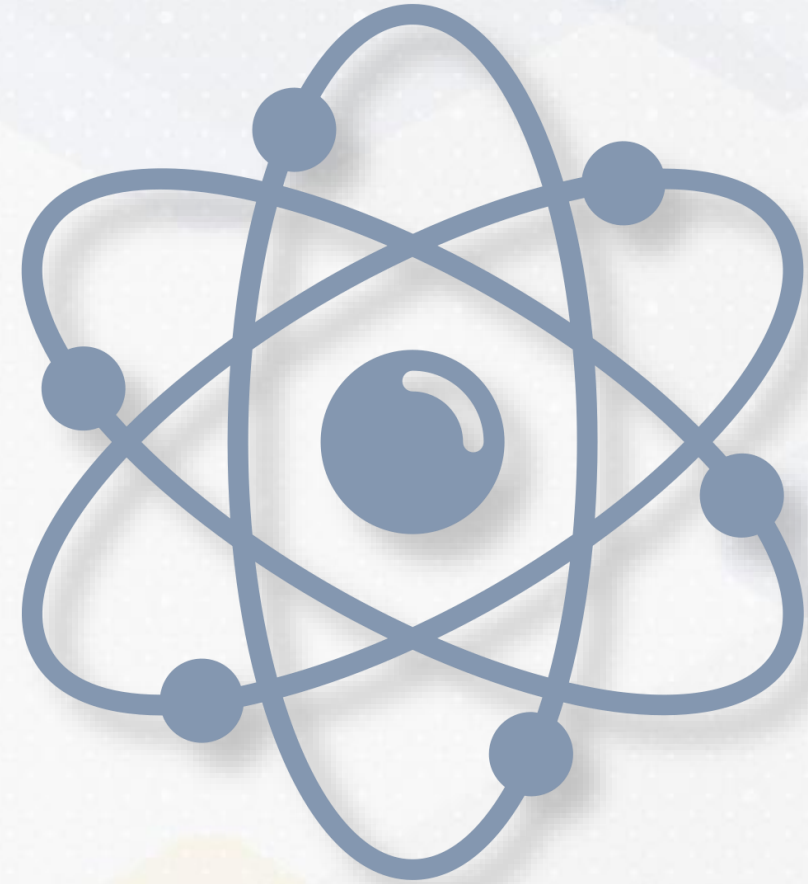
Transactions: Atomicity Property



Transactions: Atomicity Property



Transactions: Atomicity Property



Transactions: Consistency Property

```
BEGIN TRANSACTION;  
-- Withdraw $100 from account 1  
UPDATE ACCOUNT  
SET Balance = Balance - 100  
WHERE AccountID = 1;  
  
-- Deposit $100 into account 2  
UPDATE ACCOUNT  
SET Balance = Balance + 100  
WHERE AccountID = 2;  
COMMIT;
```

Transactions: Consistency Property

\$100 from Account 1 to Account 2

```
BEGIN TRANSACTION;  
-- Withdraw $100 from account 1  
UPDATE ACCOUNT  
SET Balance = Balance - 100  
WHERE AccountID = 1;  
  
-- Deposit $100 into account 2  
UPDATE ACCOUNT  
SET Balance = Balance + 100  
WHERE AccountID = 2;  
COMMIT;
```

Transactions: Consistency Property

\$100 from Account 1 to Account 2

Transaction Failure → No Partial Updates

```
BEGIN TRANSACTION;  
-- Withdraw $100 from account 1  
UPDATE ACCOUNT  
SET Balance = Balance - 100  
WHERE AccountID = 1;  
  
-- Deposit $100 into account 2  
UPDATE ACCOUNT  
SET Balance = Balance + 100  
WHERE AccountID = 2;  
COMMIT;
```


Transactions: Isolation Property

```
-- Clerk 1: Deposits $500 into account 1
BEGIN TRANSACTION;
-- Suppose it returns $1000
SELECT Balance FROM ACCOUNT WHERE AccountID = 1;
UPDATE ACCOUNT SET Balance = Balance + 500 WHERE AccountID = 1;
COMMIT;

-- Clerk 2: Deposits $300 into account 1 almost at the same time
BEGIN TRANSACTION;
-- Suppose it still returns $1000
SELECT Balance FROM ACCOUNT WHERE AccountID = 1;
UPDATE ACCOUNT SET Balance = Balance + 300 WHERE AccountID = 1;
COMMIT;
```

Transactions: Isolation Property

Clerk 1 →
\$1500

```
-- Clerk 1: Deposits $500 into account 1
BEGIN TRANSACTION;
-- Suppose it returns $1000
SELECT Balance FROM ACCOUNT WHERE AccountID = 1;
UPDATE ACCOUNT SET Balance = Balance + 500 WHERE AccountID = 1;
COMMIT;

-- Clerk 2: Deposits $300 into account 1 almost at the same time
BEGIN TRANSACTION;
-- Suppose it still returns $1000
SELECT Balance FROM ACCOUNT WHERE AccountID = 1;
UPDATE ACCOUNT SET Balance = Balance + 300 WHERE AccountID = 1;
COMMIT;
```

Transactions: Isolation Property

Clerk 1 →
\$1500

Clerk 2 →
\$1300

```
-- Clerk 1: Deposits $500 into account 1
BEGIN TRANSACTION;
-- Suppose it returns $1000
SELECT Balance FROM ACCOUNT WHERE AccountID = 1;
UPDATE ACCOUNT SET Balance = Balance + 500 WHERE AccountID = 1;
COMMIT;

-- Clerk 2: Deposits $300 into account 1 almost at the same time
BEGIN TRANSACTION;
-- Suppose it still returns $1000
SELECT Balance FROM ACCOUNT WHERE AccountID = 1;
UPDATE ACCOUNT SET Balance = Balance + 300 WHERE AccountID = 1;
COMMIT;
```


Transactions: Isolation Property

Clerk 1 →
\$1500

Clerk 2 →
\$1300

\$1500 or \$1300
Instead of \$1800

```
-- Clerk 1: Deposits $500 into account 1
BEGIN TRANSACTION;
-- Suppose it returns $1000
SELECT Balance FROM ACCOUNT WHERE AccountID = 1;
UPDATE ACCOUNT SET Balance = Balance + 500 WHERE AccountID = 1;
COMMIT;

-- Clerk 2: Deposits $300 into account 1 almost at the same time
BEGIN TRANSACTION;
-- Suppose it still returns $1000
SELECT Balance FROM ACCOUNT WHERE AccountID = 1;
UPDATE ACCOUNT SET Balance = Balance + 300 WHERE AccountID = 1;
COMMIT;
```

Transactions: Durability Property

```
BEGIN TRANSACTION;  
UPDATE ACCOUNT  
SET Balance = Balance - 500  
WHERE AccountID = 1; -- Customer withdraws $500 from ATM  
COMMIT;
```

Transactions: Durability Property

```
BEGIN TRANSACTION;  
UPDATE ACCOUNT  
SET Balance = Balance - 500  
WHERE AccountID = 1; -- Customer withdraws $500 from ATM  
COMMIT;
```

Durability

Transactions: Durability Property

```
BEGIN TRANSACTION;  
UPDATE ACCOUNT  
SET Balance = Balance - 500  
WHERE AccountID = 1; -- Customer withdraws $500 from ATM  
COMMIT;
```

Durability

Write-Ahead
Logging

Transactions: Durability Property

```
BEGIN TRANSACTION;  
UPDATE ACCOUNT  
SET Balance = Balance - 500  
WHERE AccountID = 1; -- Customer withdraws $500 from ATM  
COMMIT;
```

Durability

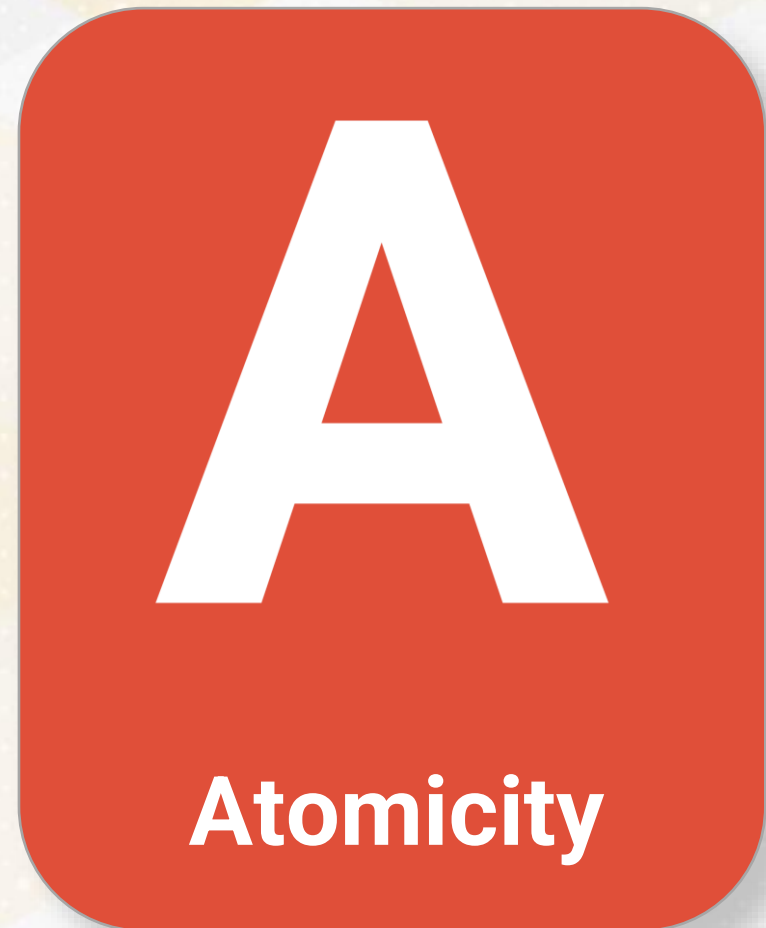
Write-Ahead
Logging

Durable
Disk

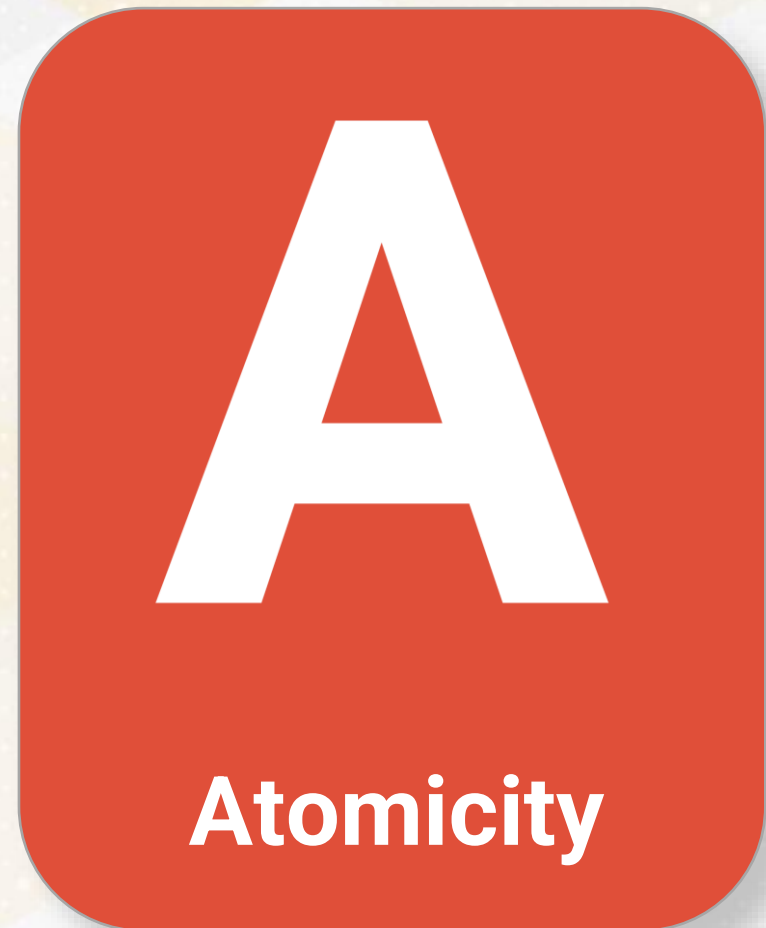
ACID Properties

The background of the slide features a white grid pattern. Overlaid on this grid are several semi-transparent squares in yellow and blue. These squares are arranged in a way that they appear to be floating or layered, with some overlapping others. The overall effect is a modern, geometric design.

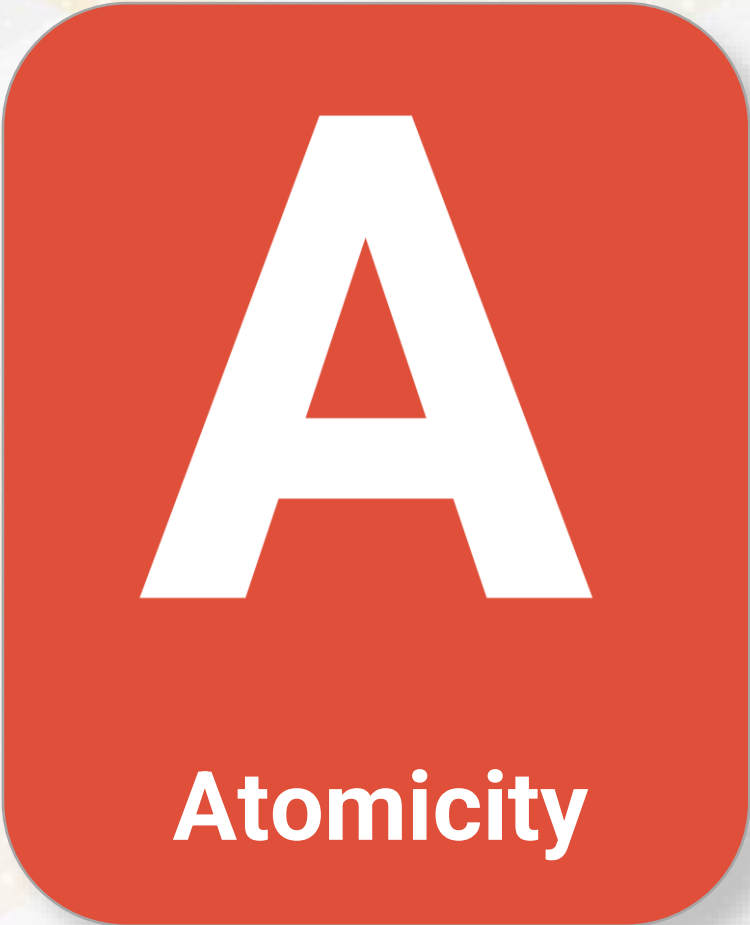
ACID Properties



ACID Properties



ACID Properties



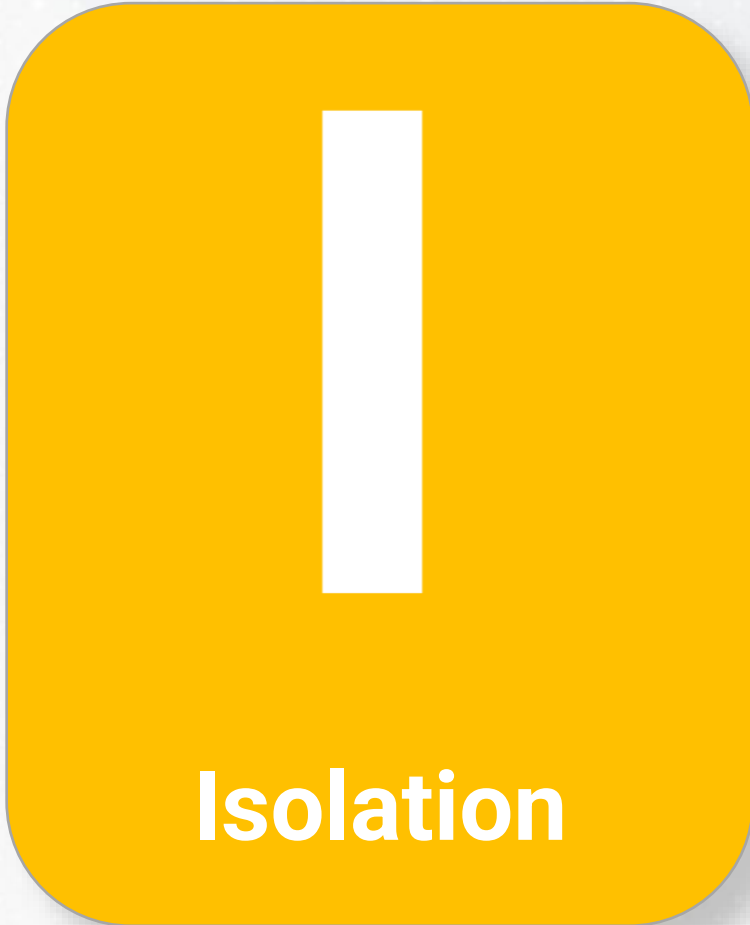
A

Atomicity



C

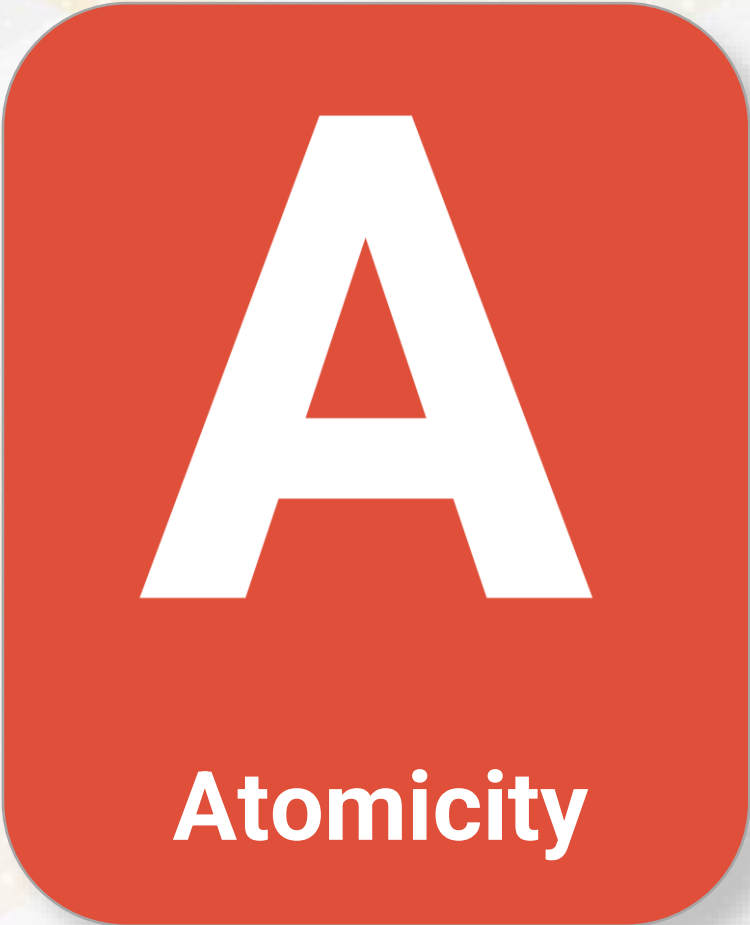
Consistency



I

Isolation

ACID Properties



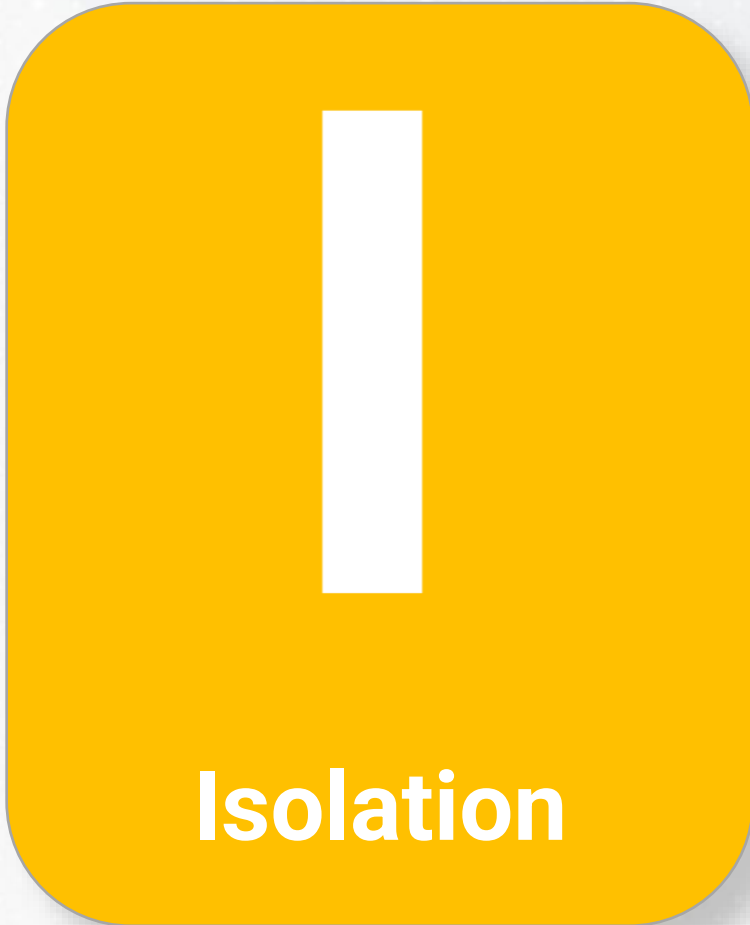
A

Atomicity



C

Consistency



I

Isolation



D

Durability

History of “ACID”



Photo: Gülay Keskin/Heidelberg Institute for Theoretical Studies (HITS)

History of “ACID”

Andreas Reuter (1983)



Photo: Gülay Keskin/Heidelberg Institute for Theoretical Studies (HITS)

History of “ACID”

Andreas Reuter (1983)

Reliable Management



Photo: Gülay Keskin/Heidelberg Institute for Theoretical Studies (HITS)

History of “ACID”

Andreas Reuter (1983)

Reliable Management

Multi-User Capability



Photo: Gülay Keskin/Heidelberg Institute for Theoretical Studies (HITS)



Transactions



Transactions

**ACID
Properties**